

Pointers

By: Menna samy

1. What is a variable, really?

A variable is not just a value sitting somewhere. It is actually two things glued together: an address (where it lives in memory) and a value (what it holds).

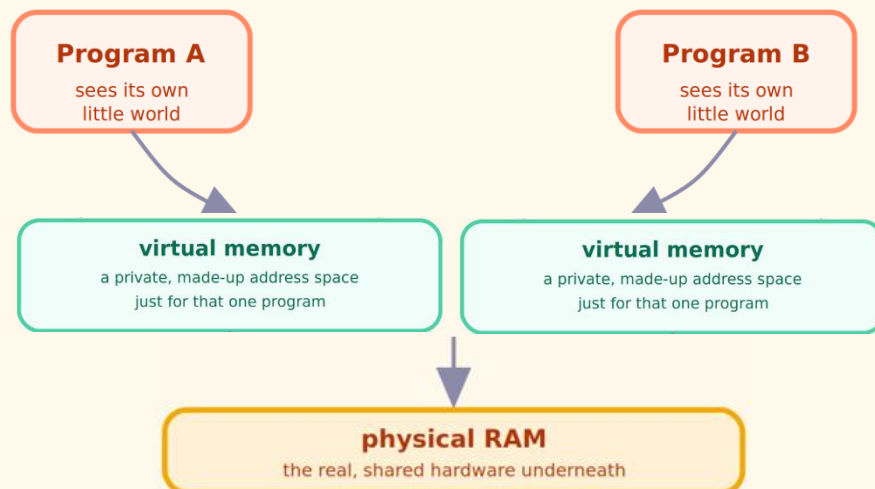


every variable is really two things living together

Virtual memory

Before we get to pointers, it helps to know where those addresses actually come from.

The RAM on your computer is physical memory, shared by everything running on the machine. But your program never touches it directly. Instead, the operating system gives every running program its own private slice, called virtual memory.



This separation matters for two big reasons:

- Isolation: if two programs run at the same time, one of them should never be able to see or mess up the other's data, just because they happen to share the same physical RAM.
- Safety: it also stops a program from accidentally (or intentionally) reaching into memory that belongs to the operating system, drivers, or other sensitive areas, which could crash the system or open up a security hole.

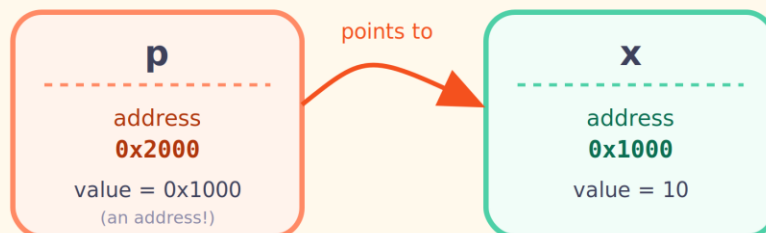
So the addresses we see as programmers, like 0x1000, are virtual addresses. They feel real to us, but the operating system quietly maps them to wherever the data truly sits in physical RAM.

2. So what exactly is a pointer?

A pointer is simply a variable whose value is an address.

```
| int* p = &x;
```

Here, *p* is a pointer, and its value is the address of *x*, not *x*'s value itself.



`int* p = &x;` ~ *p*'s value IS *x*'s address

~ p still has its own address too (it lives somewhere in memory as well) but what it stores is the address of x.

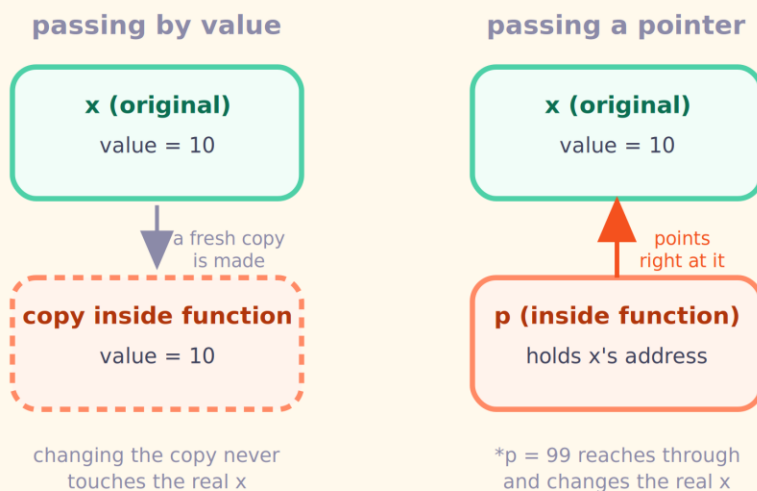
3. Ok, but why do we even need to point at a variable, when we already have it?

It looks a little redundant at first. If *x* already exists, why go through the trouble of pointing at it? A few very good reasons:

Reaching the real thing, not a copy

Normally, when you pass a variable into a function, C++ passes a copy. Anything the function does to that copy never affects the original.

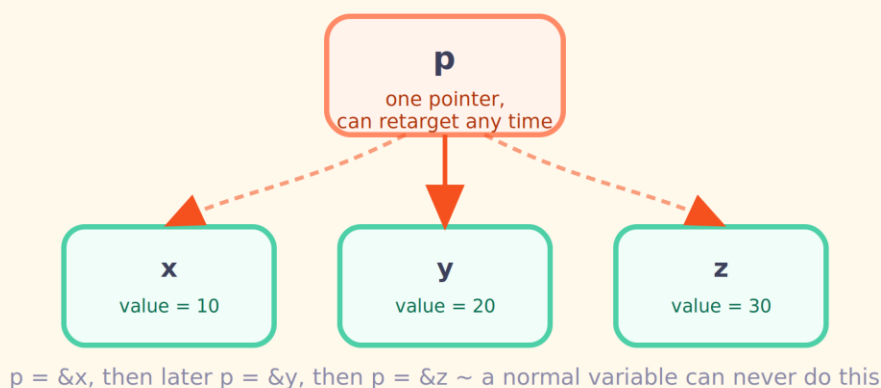
If we pass a pointer instead, the function receives the address of the original variable. So it can reach through the pointer and actually modify the real value sitting in memory.



~ this is what people mean by pass by reference using pointers.

A pointer can change who it points to

A normal variable is tied to one single piece of data. A pointer, on the other hand, can be redirected at runtime to point at different variables, one after another.



~ this lets a pointer walk across a whole set of variables, something an ordinary variable simply cannot do.

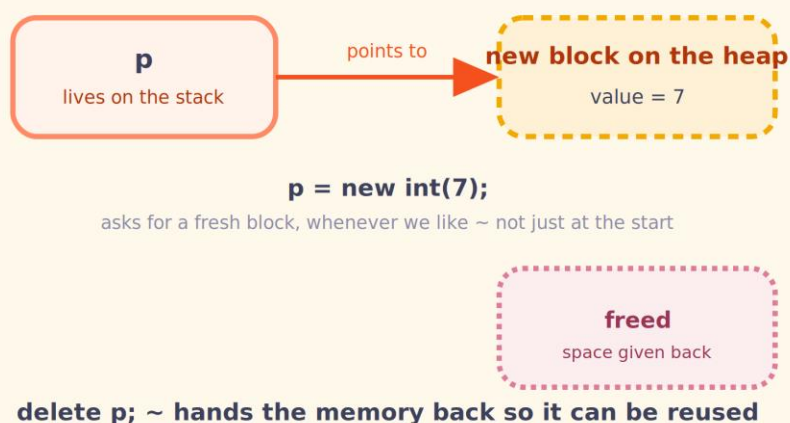
4. Dynamic memory: new and delete

So far, every variable we made existed the moment the program started that part of the code, and it disappeared automatically when it went out of scope.

Sometimes we don't know in advance how much memory we'll need, or we want memory that survives beyond a single function call. That's where dynamic memory comes in, and pointers are the only way to reach it.

```
| int* p = new int(7);
```

This line asks the operating system for a fresh block of memory on the heap, right there while the program is running, and gives us back its address in p.



```
| delete p;
```

Once we are done with that memory, we hand it back using delete. This tells the system the space is free again, so it can be reused. Forgetting to do this is called a memory leak.

~ this is the whole reason pointers matter for dynamic memory: without an address to hold on to, we would have no way to reach that block at all.